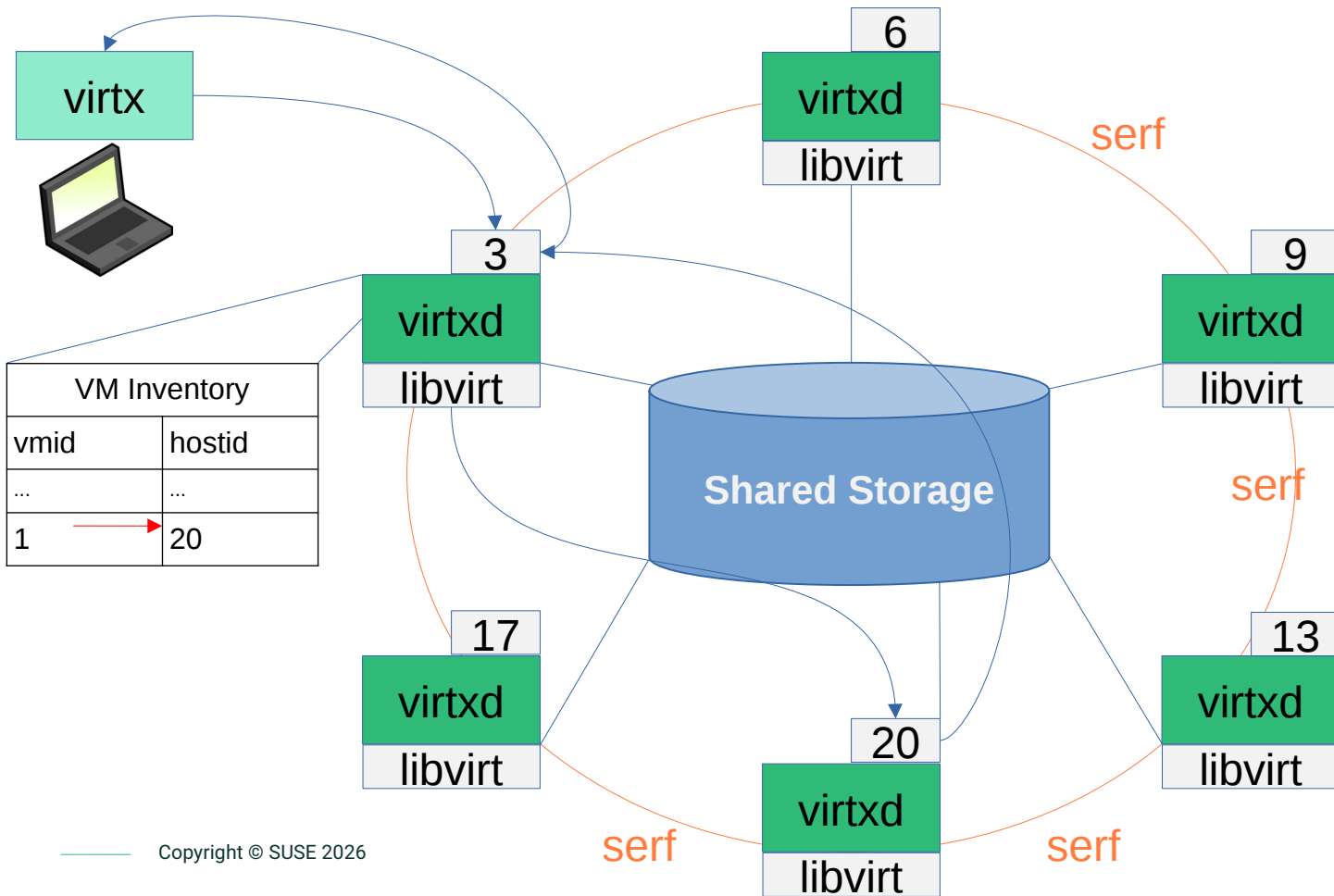


Requirements

- mainly Linux guests (but consider Windows)
- support large servers (x86_64), NUMA, large VM memory sizes (TB ranges)
- both BIOS and UEFI virtual firmware, basic VM lifecycle, live migration
- hugetlbfs for the large VMs
- shared storage only (NFS, iSCSI, FC, NVMe/TCP), no local storage
- networking just consumed as configured, vlans to separate VM groups and mgmt
- Offer a REST API, to write clients, automation, and custom add-ons
- brutally simple to configure and deploy

Desired Features

- federation of peers: all KVM nodes are equal
- inventory service: search and filter hosts and VMs in the whole cluster
- REST API service: all nodes can be used as VIRTX_API_SERVER
- NUMA pre-placement: simple on/off switch, compatible with live migration
- storage: provision and de-provision virtual disks and LUNs passed directly to VM
- resource accounting: (MhZ, MiB). Similar to vmware concept.
- simple command line client using API.
- basic host failover



Architecture

Serf: cluster membership and distribution of events, + network-based health check
SWIM (Scalable Weakly-consistent Infection-style Membership)

https://en.wikipedia.org/wiki/SWIM_Protocol

shared storage: register for VMs and hosts, sanlock lockspace and lock files, vdisks

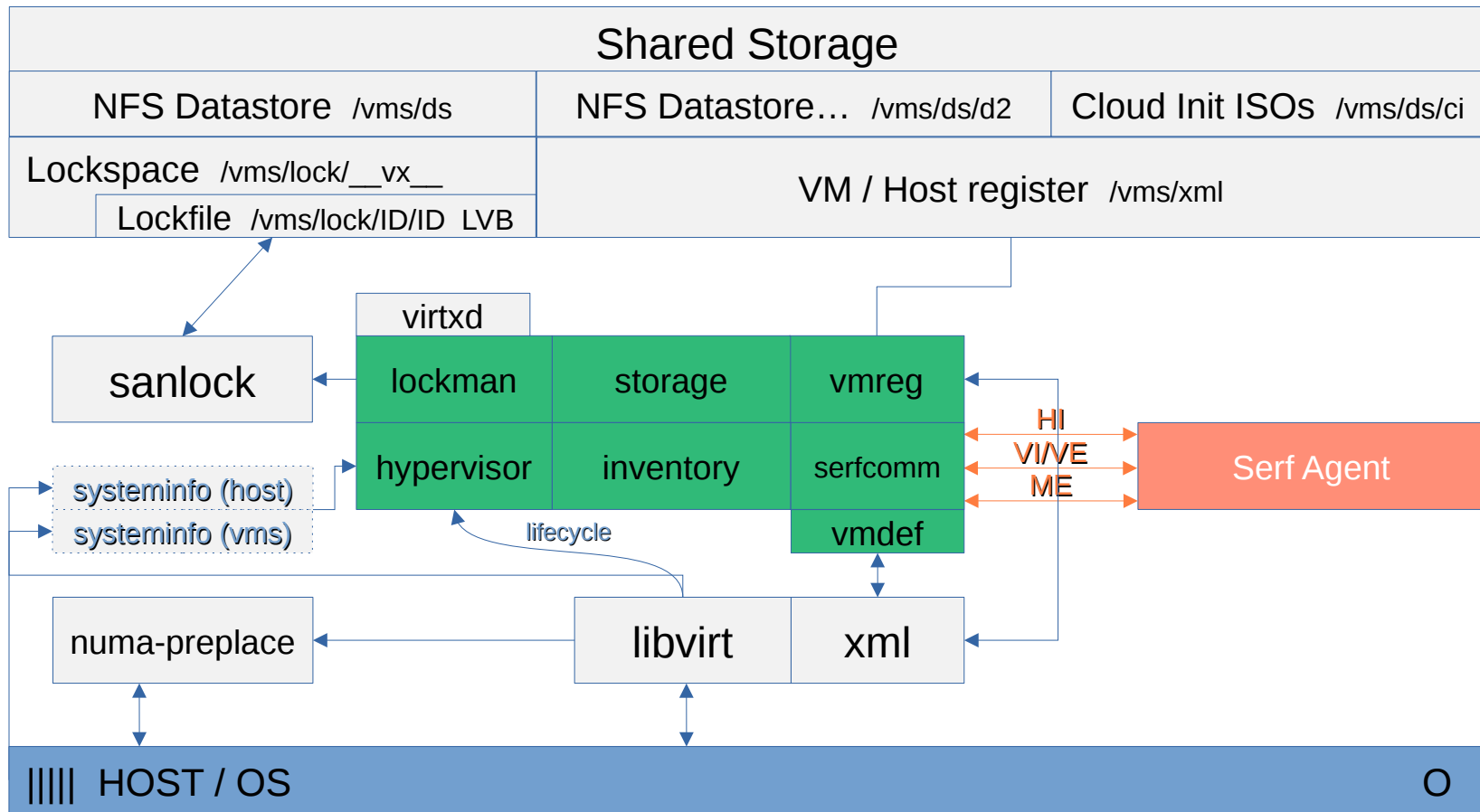
sanlock: manage shared lockspace for both vdisks and luns + storage-based health ck

virtxd: serves the REST API, comms with serf and libvirt to execute the requests

virtx client: connects to virtxd api server for REST ops

libvirt: runs the VMs with QEMU, hugepages-aware placement via **numa-preplace**

<https://www.youtube.com/watch?v=cnINrLWWFao>



Architecture Advantages

- simple automation of the KVM host deployment (out of scope of VirtX): all KVM hosts are from the cluster perspective identical. Main application is one binary.
- debuggable, can reason on the events and the sanlock leases on shared storage. Storage is the final arbiter of correctness, and self-fencing helps avoid split-brain. No etcd, complex database, scheduler or complex control plane to debug.
- no minimum number of hosts or dedicated infra nodes, the deployment can scale down to 1. REST API makes it very easy to interact with the cluster, contacting any arbitrary host.
- Libvirt and vmreg redundancy: leveraging the libvirt per-host register provides a backup for the central vmreg (...and vice versa).
- scope is mostly limited to compute, no complex storage or network abstraction layers to reconcile. Storage is passed as-is to the VMs, whose management is delegated to dedicated external storage infra. Similar with Networks.

Architecture Disadvantages

- no real workload scheduler. Workloads are assumed to be mostly static. Workload assignment is mostly manual.
- serf cluster is only “eventually consistent”, does not have strong consistency.

Failed requests should be retried.

- parallelism: any backend op that affects shared resources needs careful development (vmreg access, lock space initialization or storage resource lease creation...) to avoid race conditions, conflicts, data corruption.
- shared storage is critical – everything relies on it, so if shared storage is not accessible, the whole cluster is not operational.
- no real multi-tenancy model, or API split.

Architecture Unknowns

- scalability: needs to be tested. Should be ok in theory for the assumed limit of ~10000 Vms (100 hosts x 100 VMs), but it as yet unproven.
- ecosystem: none. Large consolidated platforms have connectors and drivers for measurements, automation, deployment, etc. Nothing exists yet for VirtX.
- sanlock locking and self-fencing: locking needs more testing and validation, and the fencing aspects are still to be carefully considered.
- VirtX is still at the prototype stage, no commercial support.

REST API

VirtX API is a REST OpenAPI 3.1 specification, maintained as a single openapi-3.1.yaml file. The API is turned into a set of Go data types called the “model”, using:

<https://github.com/openapitools/openapi-generator-cli>

with this incantation:

```
java -jar openapi-generator-cli.jar generate --type-mappings integer+int16=int16 \  
  --additional-properties=generateMarshalJSON=false,generateUnmarshalJSON=false \  
  -o client/ -g go -i openapi-3.1.yaml -t templates/
```

and some customization templates in templates/
after code generation, the model files are extracted to the virtx pkg/model.

There is a small adapter layer in pkg/model/virtx_model.go that extends some of these types with additional methods, mainly .String() and .Parse() to convert to and from a string form.

vm Everything about Virtual Machines

operationId

POST

/vms Create a new VM, optionally on a specific host. Will return the new uuid on success.

vm_create

GET

/vms Get the list of VMs in the cluster, optionally filtered (AND)

vm_list

PUT

/vms/{uuid} Update a VM changing its definition; the VM must be powered off.

vm_update

GET

/vms/{uuid} Get VM definition, state and statistics by uuid

vm_get

DELETE

/vms/{uuid} Deletes a powered-off VM, optionally deleting storage disks

vm_delete

GET

/vms/{uuid}/runstate Get VM runstate

vm_runstate_get

POST

/vms/{uuid}/runstate/boot Start a powered off VM, transitioning to running state

vm_boot

DELETE

/vms/{uuid}/runstate/boot Signal a VM to shut down gracefully, or powering off forcefully

vm_shutdown

POST

/vms/{uuid}/runstate/pause Pause a running VM, transitioning to paused state

vm_pause

DELETE

/vms/{uuid}/runstate/pause Resume a paused VM, transitioning to running state

vm_resume

POST

/vms/{uuid}/runstate/migrate Initiate a migration of a VM to another compute host

vm_migrate

GET

/vms/{uuid}/runstate/migrate Get the status information of the current migration

vm_migrate_get

DELETE

/vms/{uuid}/runstate/migrate Abort the current migration

vm_migrate_abort

PUT

/vms/{uuid}/register Attempt to explicitly register a VM between libvirt and virtx

vm_register

host Everything about Hosts

GET

/hosts List all hosts, optionally filtering (AND)

host_list

GET

/hosts/{uuid} Get detailed information about a host

host_get

SUSE

KVM Host statistics

How VirtX calculates KVM Host-level statistics:

virtxd collects system information ("system_info").

System_info includes:

"immutable" host info, only collected once, refreshed only on virtxd restart.

"mutable" host info, refetched at every system_info_loop (15 sec)

Host info: dmidecode UUID, firmware version, date, OS ID and version, hostname, cpufreq/cpuinfo_max_freq, cpu arch, vendor, model, nodes, sockets, cores, threads, tsc freq, sanlock hostid, default hugepage size.

Host resources: Normal Memory **MiB**, Huge Pages (hugetlbfs) **MiB**, CPU **MHz**

Resource fields: Total, Used, Free, ReservedVMs, UsedVMs, UsedOS, AvailableVMs

KVM Host resources

Host resources: Normal Memory **MiB**, Huge Pages (hugetlbfs) **MiB**, CPU **MHz**

Resource fields: Total, Used, Free, ReservedVMs, UsedVMs, UsedOS, AvailableVMs

HP: /proc/meminfo “HugePages_Total”, “HugePages_Free”, “Hugepagesize”(Total, Free, Used)

Mem total: libvirt, **Free:** libvirt, sysconf(_SC_AVPHYS_PAGES) or numa_node_size64()

ReservedVMs, UsedVMs: total_memory_capacity and used (sum of all vms)

AvailableVMs: Total – ReservedVMs – UsedOS

CPU total: nodes * sockets * cores * threads * max_freq **MHz**

Idle counters unreliable: **Free** calculated by difference

ReservedVMs: ((vcpus * max_freq) / 100.0) * vcpu_load_factor **MHz**

Used (libvirt): kernel_ns / (interval * 1000000) * max_freq **MHz** (+ iowait_ns + user_ns)

(libvirt “kernel_ns” = sys + irq + softirq, user = usr + nice)

Free = Total – Used **MHz**

UsedVMs: (sum_vms_cpu_utilization * max_freq / 100) **MHz**

UsedOS = Used – UsedVMs **MHz**

AvailableVMs: Total – ReservedVMs – UsedOS **MHz**

VM statistics

vms info, also refetched at every system_info_loop

domain info: uuid, name, runstate

stats: disk capacity, allocation, physical (MiB), net bandwidth (KiB/s),

vcpus, cpu utilization (% CPU) -> MHz Used, VM memory capacity (/hp), "Used" (RSS MiB)

Some stats are calculated by difference with previous loop iteration:

```
var udelta uint64 = Counter_delta_uint64(vm.cpu_time, old.cpu_time)
vm.stats.CpuUtilization = int32((udelta * 100) / (uint64(vm.Ts - old.Ts) * 1000000))
```

```
vm.stats.MhzUsed = vm.stats.CpuUtilization * int32(imm.info.MHz) / 100
```

```
var delta int64 = Counter_delta_int64(vm.net_rx, old.net_rx)
vm.stats.NetRxBw = int32((delta * 1000) / ((vm.Ts - old.Ts) * KiB))
```

Locking in VirtX

We currently have a distributed cooperative locking, used to govern access to **managed** storage resources (“disks”, ie virtual disks on NFS, or LUN devices).

Disks in VirtX are **managed** or **unmanaged**. Unmanaged disks are not locked by VirtX.

For Managed Disks the goals are:

libvirt and virtxd cooperation: libvirt and virtxd need to coordinate so they are both aware of the state of disks and whether they are currently in use (locked).

virtxd (de-)provisioning: VirtX takes care of preparing a disk for use, creating it if necessary or taking over an existing disk. When a VM is deleted, VirtX can optionally also de-provision the managed disks.

VM disk ownership: each VM fully owns its managed disks. This means that two VMs cannot use the same managed disk, even if the other VM is shutdown. No “ping-pong”.

Locking in VirtX: libvirt cooperation

libvirt does not offer a public locking API.

It has an internal API with two drivers (lockd, sanlock). Lockd is for internal use (virtlockd).

By using sanlock referring to the same lockspace and resources we solve the problem for the major part, with some edge case caveats.

VirtX will create a resource file (paxos lease file) any time it provisions a new managed disk, and it will delete the resource file when it de-provisions the managed disk.

VirtX will add <lease> elements to the libvirt XML definition of the VM referring to the correct lockspace, resource key and resource path, so that libvirt can acquire the lease on the resource.

This way, VirtX and libvirt can coordinate on when a resource is accessible.

Locking in VirtX: initialization

On startup, virtxd checks if the lockspace delta lease file is already existing and properly initialized; if not, it will create a temporary directory, fully initialize the delta lease file, and then attempt a rename operation to the empty final destination directory, so that parallel attempts to initialize are captured by EEXIST or ENOTEMPTY on the rename.

Then it instructs the local sanlock daemon to join the lockspace if not already joined, identifying the host with a unique `host_id` to be marked in the shared delta lease file.

This `host_id` can be mapped initially from the host uuid to a (1-2000) `host_id`, and on collision a linear search of the space is done to find a free slot. Once the lockspace is successfully joined, the `host_id` is cached in vmreg for future use. It can also be “manually” configured in the cache if desired.

New resource creation (paxos lease file) use the same algorithm with the final rename as per delta lease files, to ensure that a resource file is either visible and fully initialized, or not visible (never visible with partly-initialized contents).

Locking in VirtX: provisioning and de-provisioning

When a VM is newly created or updated, all disks that are marked as MANAGED go through a provisioning step.

a resource is created and acquired for each of those disks, and is initialized with a LVB (lock value block) containing the UUID of the owning VM.

For virtual disks, VirtX will optionally create a new disk (qemu-img create), .qcow2 or .raw are supported, or it can claim existing disks. The disk can be marked as “thin” or “thick” and will be provisioned accordingly, or for existing disks, the provisioning mode is detected.

For LUNs, VirtX will attempt to efficiently clear the disk, using blkdiscard if supported, or wipefs and dd. The provisioning mode cannot be reliably set and currently not even detected from VirtX in all cases (* improvement).

After the disk is fully provisioned, the resource lease will be released.

For de-provisioning, it is all very similar, with the disk being optionally deleted / wiped, and the resource lease file deleted, all while holding the lease.

Locking in VirtX: VM disk ownership

The LVB of each resource file contains the UUID of the owning VM.

This is checked under lease each time VirtX attempts to acquire the lease, and if the UUID does not match the VM for which it is executing the action, it will immediately fail and release the lease.

This makes it possible to implement the ownership concept, and also helps provide better error messages for problem solving (“cannot acquire lease: disk is owned by VM xxxxxx”).

This concept is not present in libvirt.

If we create or update a VM in VirtX with the same managed disk owned by another VM we catch the issue there, so it should not be a problem in most cases, as long as any access to libvirt is mediated by VirtX. But having libvirt native support would make it more reliable.

Implementation

<https://github.com/openSUSE/virtx> - OpenSUSE public project

API still unstable. Some immediately actionable missing items:

- test infrastructure (mainly to avoid regressions) - ongoing
- better networking (vlan) support
- Gold Images support of some kind
- better live migration statistics
- security review, API security
- VM resource reservation/limits and enforcement
- console access via API (text, vnc)
- minimal failover and fencing support on host failure
- Some external UI (maybe a cockpit plugin)?
- VM import jobs (from VMWare, other libvirt/QEMU platforms?)